

S I S T E M A D E E - C O M M E R C E M U L T I P L A T A F O R
M A

Documentacion del Sistema JO-Shop

Arquitectura, Modulos y Funcionalidades

Backend: Express.js + Prisma + MongoDB

Frontend: Next.js 15.5 | Mobile: React Native

Tiempo Real: Pusher | GPS Tracking Integrado

Tabla de Contenidos

1. Introduccion y Vision General.....	1
1.1 Arquitectura del Sistema.....	1
2. JO-backend-shop (API Backend).....	2
2.1 Estructura del Proyecto.....	2
2.2 Autenticacion y Autorizacion.....	3
2.3 Base de Datos (Prisma + MongoDB).....	3
2.4 Rutas de Chat.....	3
2.4.1 Chat de Ordenes.....	4
2.4.2 Chat Administrativo.....	4
2.5 Rutas de Configuracion.....	4
2.6 Rutas de GPS Tracking.....	4
3. JO-frontend-shop (Tienda Web).....	5
3.1 Estructura y Navegacion.....	5
3.2 Contextos de React.....	5
3.2.1 ConfigContext.....	6
3.2.2 PusherContext.....	6
3.3 Sistema de Chat en el Frontend.....	6
3.4 Pagina de Tracking GPS.....	6
3.5 Sistema de Configuracion Visual.....	7
3.6 Loader y Pantalla de Carga.....	7
4. JO-Shop (App Movil - Compradores).....	7
4.1 Pantalla de Pedidos y Chat.....	8
4.2 Canales de Pusher.....	8
4.3 Flujo de Notificaciones.....	8
5. JO-Delivery (App Movil - Repartidores).....	8
5.1 Pantalla de Ordenes de Entrega.....	9
5.2 GPS Tracking en Tiempo Real.....	9

5.3 Canales de Pusher y Comunicacion.....	9
6. Sistema Pusher en Tiempo Real.....	9
6.1 Canales de Pusher.....	10
6.2 Eventos de Pusher.....	10
6.3 Flujo de un Mensaje de Chat.....	10
7. Sistema de Configuracion y Apariencia.....	11
7.1 Configuraciones Disponibles.....	11
7.2 Flujo de Aplicacion de Colores.....	11
8. Flujo Completo de un Pedido.....	11
8.1 Creacion del Pedido.....	12
8.2 Asignacion y Preparacion.....	12
8.3 Entrega en Tiempo Real.....	12
8.4 Finalizacion del Pedido.....	12
9. Seguridad y Buenas Practicas.....	13

1. Introduccion y Vision General

JO-Shop es un sistema de e-commerce multiplataforma disenado para gestionar una tienda en linea completa, abarcando desde la creacion de productos y gestion de pedidos hasta el seguimiento de entregas en tiempo real y la comunicacion directa entre compradores, vendedores y repartidores. El ecosistema esta compuesto por cuatro repositorios principales que trabajan de forma coordinada a traves de una API REST centralizada y el servicio de tiempo real Pusher.

El sistema fue concebido como una solucion integral que elimina la necesidad de multiples herramientas desconectadas. Toda la logica de negocio reside en el backend (JO-backend-shop), mientras que tres interfaces de usuario distintas consumen la misma API: un frontend web para clientes (JO-frontend-shop), una aplicacion movil para compradores (JO-Shop) y una aplicacion movil para repartidores (JO-Delivery). Cada plataforma mantiene su propia identidad visual configurable desde el panel de administracion.

1.1 Arquitectura del Sistema

La arquitectura sigue un modelo cliente-servidor clasico con una capa de tiempo real superpuesta. El backend funciona como un servidor Express.js que expone endpoints RESTful y se comunica con MongoDB a traves de Prisma ORM. Las tres plataformas cliente se conectan al backend via HTTP para operaciones CRUD y a Pusher para actualizaciones en tiempo real. Este diseno permite que los cambios en un componente se reflejen instantaneamente en todos los demas sin necesidad de polling.

La base de datos es MongoDB, accesada a traves del adapter oficial de Prisma para MongoDB (@prisma/adapter-mongodb). El esquema de datos utiliza modelos de Prisma que se mapean a colecciones de MongoDB, lo que permite flexibilidad en la estructura de documentos mientras se mantiene la seguridad tipica de un ORM. La configuracion del sistema (nombre de tienda, colores, logos) se almacena en un modelo key-value llamado SystemConfig.

Componente	Tecnologia	Funcion Principal
JO-backend-shop	Express.js + Prisma + MongoDB	API REST, logica de negocio, base de datos
JO-frontend-shop	Next.js 15.5.15 (App Router)	Tienda web para clientes
JO-Shop	React Native	App movil para compradores
JO-Delivery	React Native	App movil para repartidores
Pusher	WebSocket Service	Tiempo real (chat, notificaciones, tracking)

2. JO-backend-shop (API Backend)

El backend es el corazon del sistema, alojado en Vercel y construido con Express.js. Toda la logica de negocio, autenticacion, almacenamiento de archivos y comunicacion en tiempo real se gestiona desde este servicio. Se conecta a MongoDB usando Prisma ORM con el adapter nativo para MongoDB, lo que permite consultas tipadas y un esquema de datos bien definido.

2.1 Estructura del Proyecto

El proyecto sigue una estructura de carpetas modular tipica de Express.js. Las rutas se organizan por dominio funcional en el directorio `src/routes/`, cada una con su middleware de autenticacion apropiado. El middleware de autenticacion (`src/middleware/auth.js`) valida tokens JWT y la cabecera X-Platform para identificar que plataforma esta haciendo la peticion. La configuracion de Prisma esta en `prisma/schema.prisma` y las variables de entorno se manejan via `.env`.

Directorio	Descripcion
<code>src/routes/</code>	Rutas API (auth, orders, products, chats, config, tracking)
<code>src/middleware/</code>	Auth, hasRole, validPlatform, passwordCheck
<code>src/lib/</code>	Utilidades compartidas (prisma.js, helpers)
<code>prisma/</code>	Schema de Prisma, migraciones, seed
<code>src/routes/chats.js</code>	Endpoints de chat (admin + ordenes)
<code>src/routes/config.js</code>	Configuracion del sistema, upload de logos
<code>src/routes/tracking.js</code>	Endpoints de GPS tracking

2.2 Autenticacion y Autorizacion

El sistema utiliza JSON Web Tokens (JWT) para la autenticacion. Cuando un usuario inicia sesion, el backend genera un token JWT que incluye el `userId`, roles y la plataforma de origen. Los clientes deben enviar este token en la cabecera `Authorization: Bearer <token>` en cada peticion protegida. El middleware `authenticate` decodifica el token y adjunta `req.user` con la informacion del usuario. Adicionalmente, existe un middleware `hasRole` que verifica si el usuario tiene un rol especifico (admin, delivery, etc.).

Una capa adicional de seguridad es la cabecera `X-Platform`, que cada cliente debe enviar con su identificador de plataforma. Los valores validos son: `frontend-shop` (Next.js), `landingpage`, `app-shop` (React Native compradores) y `app-delivery` (React Native repartidores). El backend usa esta cabecera para validar la procedencia de las peticiones y para emitir eventos de Pusher en los canales

correctos. La contraseña de servicio entre backend (BACKEND_SERVICE_PASSWORD) protege endpoints internos.

2.3 Base de Datos (Prisma + MongoDB)

El modelo de datos se define en `prisma/schema.prisma` usando Prisma ORM con el adapter `@prisma/adapter-mongodb`. Aunque MongoDB es una base de datos NoSQL, Prisma impone un esquema rígido que valida la estructura de los documentos antes de insertarlos. Los modelos principales incluyen `User`, `Product`, `Order`, `OrderItem`, `ChatMessage`, `Category`, `Store` y `SystemConfig`. Cada modelo tiene sus relaciones definidas, permitiendo consultas con `include` y `select`.

El modelo `SystemConfig` es especialmente relevante porque funciona como un almacén key-value donde se guardan todas las configuraciones del sistema. Cada fila tiene un key único (como `shop_name`, `shop_logo_url`, `primary_color`, `accent_color`, `shop_loader_url`, `multi_store`, `delivery_name`, `delivery_logo_url`) y un value de tipo texto. Esto permite que la configuración se actualice dinámicamente sin cambiar el esquema de la base de datos. La semilla inicial (`prisma/seed.js`) crea únicamente el registro `multi_store_mode` con valor `false`.

2.4 Rutas de Chat

El módulo de chat es uno de los más complejos del backend, implementado en `src/routes/chats.js`. Soporta dos modalidades de chat: chat de ordenes (entre comprador y vendedor/admin) y chat administrativo (entre el admin y cualquier usuario de cualquier plataforma). Ambos tipos de chat emiten eventos a través de Pusher para que los destinatarios reciban los mensajes en tiempo real.

2.4.1 Chat de Ordenes

El chat de ordenes está vinculado a un pedido específico. Los endpoints son `GET /chats/orders/{orderId}/messages` para obtener el historial y `POST /chats/orders/{orderId}/messages` para enviar un nuevo mensaje. Al enviar un mensaje, el backend lo guarda en la base de datos y emite un evento `order-message` al canal privado del usuario propietario del pedido: `private-user-{order.userId}`. El evento incluye los datos del mensaje y la información del pedido para que el cliente pueda mostrar una notificación contextual con el número de orden.

2.4.2 Chat Administrativo

El chat administrativo permite al admin comunicarse directamente con cualquier usuario del sistema, independientemente de si tiene un pedido activo. El endpoint `GET /chats/admin/messages?recipientId=X&recipientPlatform=Y` obtiene el historial de conversación con un usuario específico, y `POST /chats/admin/messages` envía un nuevo mensaje. Al enviar, el

backend emite el evento order-message al canal del destinatario usando su ID y plataforma. La plataforma del destinatario es crucial para construir correctamente el ID del canal de Pusher.

2.5 Rutas de Configuracion

Las rutas de configuracion (src/routes/config.js) gestionan la apariencia y comportamiento del sistema. GET /config devuelve todas las configuraciones como un objeto plano key-value y es publico (no requiere autenticacion), ya que el frontend lo necesita para aplicar el tema antes de que el usuario inicie sesion. PUT /config actualiza configuraciones y requiere rol de admin. Los endpoints de upload usan multer para procesar archivos en memoria y @vercel/blob para almacenarlos en la nube de Vercel.

Existen configuraciones independientes para la tienda (shop_name, shop_logo_url, shop_loader_url, primary_color, accent_color) y para la app de delivery (delivery_name, delivery_logo_url, delivery_primary_color, delivery_accent_color). Los endpoints de upload soportan un maximo de 2MB y formatos JPG, PNG, WebP y SVG. Las URLs de las imagenes subidas se almacenan en SystemConfig y se sirven directamente desde Vercel Blob.

2.6 Rutas de GPS Tracking

El modulo de tracking permite a los repartidores compartir su ubicacion en tiempo real. El endpoint POST /tracking/location recibe las coordenadas GPS del repartidor y las emite a traves de Pusher al canal privado-order-{orderId}. Los clientes suscritos a este canal reciben actualizaciones de posicion cada 15 segundos. El tracking se gestiona desde la app JO-Delivery usando el servicio tracking.js, que implementa watchPosition del navegador geolocation API con un intervalo de 15 segundos y una distancia minima de 10 metros entre actualizaciones.

3. JO-frontend-shop (Tienda Web)

El frontend web esta construido con Next.js 15.5.15 usando el App Router de Next.js. Es la interfaz principal para los clientes que compran desde el navegador, ofreciendo una experiencia responsiva que funciona tanto en desktop como en movil. Utiliza React Server Components donde es posible y Client Components solo cuando se necesita interactividad (formularios, Pusher, estados locales).

3.1 Estructura y Navegacion

La aplicacion utiliza un layout principal con una barra de navegacion superior (Header) y una barra de navegacion inferior fija (BottomNav) con cuatro pestañas: Inicio (/), Carrito (/cart), Pedidos (/my-orders) y Perfil (/profile). La pagina de inicio muestra productos en un grid con busqueda y filtrado por categorias. El carrito permite seleccionar cantidad y proceder al checkout. Los pedidos muestran un historial con estados y un modal de chat integrado.

Ruta	Componente	Funcionalidad
/	page.tsx	Catalogo de productos, busqueda, categorias, footer
/cart	CartPage	Carrito de compras, checkout
/my-orders	page.tsx	Historial de pedidos, chat por pedido
/tracking/[id]	page.tsx	Seguimiento GPS del repartidor
/profile	ProfilePage	Perfil de usuario
/settings/*	SettingsPages	Panel de administracion (solo admin)

3.2 Contextos de React

La aplicacion utiliza cuatro contextos principales que proveen datos y funciones a todos los componentes hijos. AuthContext gestiona la autentificacion del usuario (login, logout, token JWT, roles). ConfigContext obtiene y sincroniza la configuracion del sistema (nombre, logo, colores, loader). PusherContext maneja la conexion a Pusher y las suscripciones a canales. NotificationContext gestiona las notificaciones locales (in-app), mostrando badges y alertas cuando llegan nuevos mensajes o actualizaciones de pedidos.

3.2.1 ConfigContext

El ConfigContext es especialmente interesante porque realiza multiples funciones criticas. Al montarse, hace un GET /config para obtener toda la configuracion del sistema. Luego, sincroniza los colores como propiedades CSS personalizadas (--primary, --accent, --primary-gradient, etc.) directamente en el document.documentElement. Tambien persiste la configuracion en localStorage bajo la clave joshop_theme para que el loader muestre los colores correctos antes de que React hidrate. Finalmente, despacha el evento theme-ready que el script del loader (en layout.tsx) escucha para desaparecer la pantalla de carga. Incluye funciones para subir logos (uploadLogo), imagenes de loader (uploadLoader), y actualizar configuraciones (updateConfig).

3.2.2 PusherContext

El PusherContext inicializa la conexion a Pusher usando las credenciales configuradas (app_id, key, cluster). Al conectarse, suscribe automaticamente al canal privado del usuario actual: private-user-{userId}. En este canal escucha el evento order-message que el backend emite cada vez que hay un nuevo mensaje de chat o una actualizacion de pedido relevante. Cuando recibe un evento, despacha un evento personalizado pusher:order-message que el GlobalPusherListener captura para crear notificaciones. Tambien expone las funciones subscribeToOrderChannel(orderId) y

`unsubscribeFromOrderChannel(orderId)` para suscribirse a canales especificos de pedidos (`private-order-{orderId}`) usados en la pagina de tracking.

3.3 Sistema de Chat en el Frontend

El chat se integra en la vista de mis pedidos (`my-orders/page.tsx`). Cada pedido tiene un boton que abre un modal de chat donde se muestra el historial de mensajes y un campo para enviar nuevos mensajes. El componente implementa polling (consulta periodica al backend) combinado con Pusher para recibir mensajes en tiempo real. Cuando el usuario envia un mensaje, se hace un POST a `/chats/orders/{orderId}/messages` y simultaneamente el backend emite el evento a traves de Pusher para que la otra parte lo reciba sin necesidad de recargar.

3.4 Pagina de Tracking GPS

La pagina de tracking (`/tracking/[id]/page.tsx`) muestra la ubicacion en tiempo real del repartidor asignado a un pedido. Utiliza un iframe de OpenStreetMap centrado en las coordenadas del repartidor en lugar de un componente de mapa interactivo. La eleccion de un iframe se debio a problemas con los tiles de Leaflet que se bloqueaban por politicas de CORS. La pagina se suscribe al canal `private-order-{orderId}` a traves de `PusherContext` y actualiza la posicion del mapa cada vez que recibe nuevas coordenadas. Incluye un bottom sheet que muestra los detalles del pedido, el estado actual y la informacion del repartidor.

El ciclo de vida de la suscripcion a Pusher se maneja correctamente con `useEffect`: al montar el componente se llama `subscribeToOrderChannel(orderId)`, y en la funcion de limpieza del `useEffect` se llama `unsubscribeFromOrderChannel(orderId)`. Esto garantiza que no haya suscripciones fantasmas cuando el usuario navega fuera de la pagina de tracking.

3.5 Sistema de Configuracion Visual

El panel de apariencia (`settings/appearance/page.tsx`) permite al administrador personalizar completamente la tienda. Los campos editables incluyen el nombre del sistema, el color primario (con selector de color y entrada hex), el color secundario, el logo del sistema y la imagen de carga (loader). Los cambios se guardan inmediatamente a traves de `PUT /config` y el logo/loader se suben via `POST /config/upload-logo` y `POST /config/upload-loader` respectivamente.

La imagen del loader es una funcionalidad que permite personalizar la pantalla de carga inicial. Si se sube una imagen (JPG, PNG, WebP, SVG o GIF, maximo 2MB), esta se muestra en el centro de la pantalla con un borde animado giratorio. Si no se sube ninguna imagen, el loader muestra las dos primeras letras del nombre de la tienda con la misma animacion de borde. La URL de la imagen se persiste en `localStorage` junto con la configuracion de colores para que el loader la muestre instantaneamente en la proxima visita, antes de que React cargue.

3.6 Loader y Pantalla de Carga

La pantalla de carga es una overlay HTML pura que se renderiza en `layout.tsx` antes de que React hidrate la aplicacion. Se compone de un `div#theme-loader` con fondo blanco, centrado en pantalla con `z-index 99999`. Dentro tiene un `div.loader-logo` de `64x64px` con borde redondeado de `18px`. El borde animado se logra con un pseudo-elemento `::before` que usa un `conic-gradient` rotando a traves de la animacion `loaderBorderSpin` (`1.2s` lineal infinito). El loader desaparece cuando `ConfigContext` despacha el evento `theme-ready`, que activa la clase CSS `data-theme-ready` en el HTML, aplicando una transicion de opacidad de `0.4s`. Si algo falla, el loader se oculta automaticamente despues de 4 segundos como medida de seguridad.

4. JO-Shop (App Movil - Compradores)

JO-Shop es la aplicacion movil para compradores, construida con React Native. Permite a los usuarios navegar productos, hacer pedidos, pagar y comunicarse con el vendedor a traves de chat integrado con Pusher. La app se conecta al mismo backend que el frontend web, compartiendo la API REST y los canales de Pusher.

4.1 Pantalla de Pedidos y Chat

La pantalla principal de pedidos (`OrdersScreen.js`) muestra la lista de pedidos del usuario con su estado actual (pendiente, confirmado, en camino, entregado, cancelado). Cada pedido tiene acciones contextuales que abren un modal de confirmacion con dos botones: Cancelar y Ver. El boton Ver navega a la pantalla de chat del pedido correspondiente. Antes de navegar, se establece la pestana activa en `my_deliveries` para que al presionar atras, el usuario vuelva a la lista de pedidos en la pestana correcta. Las notificaciones de nuevos mensajes se manejan con `Alert.alert` que al aceptar navega al chat.

4.2 Canales de Pusher

JO-Shop se conecta a Pusher usando credenciales compartidas y suscribe al canal de presencia del usuario: `presence-chat-app-shop-{userId}`. Este canal indica al sistema que el usuario esta activo en la app movil de compradores. La informacion del canal de presencia se usa por el backend y el frontend web para saber en que plataforma esta conectado cada usuario, lo que permite dirigir los mensajes al canal correcto.

4.3 Flujo de Notificaciones

Cuando el backend emite un evento `order-message` al canal privado del usuario, la app de JO-Shop lo recibe a traves de su conexion Pusher. Si la app esta en primer plano, muestra un `Alert.alert` con

los detalles del mensaje y un boton para navegar al chat correspondiente. Si la app esta en segundo plano, el sistema operativo maneja la notificacion push. El dismiss del alert en segundo plano navega al chat directamente usando la referencia de navegacion.

5. JO-Delivery (App Movil - Repartidores)

JO-Delivery es la aplicacion movil para repartidores, tambien construida con React Native. Permite a los repartidores ver sus asignaciones, aceptar entregas, navegar a las direcciones de entrega, compartir su ubicacion GPS en tiempo real y comunicarse con compradores y administradores a traves de chat.

5.1 Pantalla de Ordenes de Entrega

La pantalla principal (DeliveryOrdersScreen.js) muestra los pedidos asignados al repartidor. Utiliza un sistema de pestanas con tabs para separar pedidos pendientes de entregas en progreso. Al pulsar sobre un pedido, se muestra un ConfirmModal con opciones de Cancelar y Ver, similar al patron usado en JO-Shop. La integracion con el chat permite al repartidor comunicarse directamente con el comprador o el administrador durante la entrega.

5.2 GPS Tracking en Tiempo Real

El servicio de tracking (src/services/tracking.js) es uno de los componentes mas criticos de JO-Delivery. Implementa dos mecanismos de seguimiento GPS: un watchPosition del navegador geolocation API para actualizaciones continuas (cada cambio de posicion significativo) y un setInterval de 15 segundos como respaldo. Las coordenadas se envian al backend via POST /tracking/location junto con el orderId y el deliveryId. El backend emite estas coordenadas al canal privado-order-{orderId} de Pusher para que los clientes puedan ver la ubicacion del repartidor en tiempo real.

Se implementa una distancia minima de 10 metros entre actualizaciones consecutivas para evitar enviar datos excesivos cuando el repartidor esta detenido. El servicio expone tres funciones principales: startTracking(orderId) inicia el seguimiento para un pedido especifico, stopTracking() detiene toda actividad de tracking y limpia los intervalos, y las coordenadas se envian con timestamp y precision para que el frontend pueda calcular la velocidad y direccion del repartidor.

5.3 Canales de Pusher y Comunicacion

JO-Delivery se conecta a Pusher y suscribe al canal de presencia presence-chat-app-delivery-{userId}. Esto permite al sistema saber que el repartidor esta activo. La app recibe eventos de Pusher para notificaciones de nuevos pedidos asignados, mensajes de chat y actualizaciones de

estado de ordenes. La integración con el chat permite comunicación bidireccional con el comprador a través del chat vinculado al pedido.

6. Sistema Pusher en Tiempo Real

Pusher es el servicio centralizado de comunicación en tiempo real que conecta todos los componentes del sistema. Permite que los mensajes de chat, las actualizaciones de pedidos y las coordenadas GPS se transmitan instantáneamente entre plataformas sin necesidad de que los clientes hagan consultas periódicas al backend.

6.1 Canales de Pusher

El sistema utiliza tres tipos de canales de Pusher, cada uno con un propósito específico:

Tipo de Canal	Patrón de Nombre	Uso
Private (usuario)	private-user-{userId}	Notificaciones y mensajes directos
Private (pedido)	private-order-{orderId}	Tracking GPS y updates de pedido
Presence	presence-chat-{platform}-{userId}	Indicador de conexión del usuario

Los canales privados requieren autenticación, que el backend maneja a través del endpoint de autorización de Pusher. Los canales de presencia extienden los canales privados y adicionalmente exponen información sobre quienes están conectados. Los IDs compuestos de los canales de presencia (presence-chat-{platform}-{userId}) permiten que un mismo usuario pueda estar conectado simultáneamente desde múltiples plataformas, cada una en su propio canal.

6.2 Eventos de Pusher

El evento principal del sistema es order-message, emitido por el backend cada vez que ocurre una acción relevante. Este evento se transmite a través del canal privado del usuario destino y contiene información sobre el tipo de evento (nuevo mensaje de chat, actualización de estado, etc.), los datos del mensaje y la información del pedido asociado. En el frontend web, el GlobalPusherListener captura estos eventos y los convierte en notificaciones in-app con la función addNotification(). En las apps móviles, los eventos se capturan directamente en los listeners de Pusher.

6.3 Flujo de un Mensaje de Chat

Cuando un usuario envia un mensaje, el flujo completo es: (1) El cliente hace un POST a /chats/orders/{orderId}/messages o /chats/admin/messages con el contenido del mensaje. (2) El backend guarda el mensaje en la base de datos MongoDB. (3) El backend emite un evento order-message al canal privado del destinatario usando Pusher. (4) El destinatario recibe el evento en tiempo real a traves de su conexion Pusher activa. (5) La aplicacion del destinatario muestra una notificacion y actualiza la interfaz de chat. Todo este proceso ocurre en milisegundos, proporcionando una experiencia de chat instantaneo.

7. Sistema de Configuracion y Apariencia

El sistema de configuracion es uno de los modulos mas versatiles del ecosistema. Permite personalizar completamente la identidad visual de cada plataforma desde un panel centralizado, sin necesidad de modificar codigo. Todas las configuraciones se almacenan en la tabla SystemConfig de MongoDB como pares clave-valor, lo que permite agregar nuevas configuraciones sin alterar el esquema de la base de datos.

7.1 Configuraciones Disponibles

Clave	Descripcion	Valor por Defecto
shop_name	Nombre de la tienda	JO-Shop
shop_logo_url	URL del logo del sistema (Vercel Blob)	
shop_loader_url	URL de la imagen de carga (Vercel Blob)	
primary_color	Color primario en formato hex	#FF6B35
accent_color	Color secundario en formato hex	#E94560
multi_store	Modo multi-tienda activado	false
delivery_name	Nombre de la app de delivery	JO-Delivery
delivery_logo_url	Logo de la app de delivery	
delivery_primary_color	Color primario de delivery	#FF6B35
delivery_accent_color	Color secundario de delivery	#E94560

7.2 Flujo de Aplicacion de Colores

La aplicacion de colores sigue un flujo de tres etapas para garantizar la maxima velocidad de carga. Primero, al cargar la pagina por primera vez, un script inline en layout.tsx lee la configuracion cacheada del localStorage y aplica los colores como variables CSS personalizadas (--primary, --accent, --primary-gradient, etc.) antes de que React se hidrate. Segundo, cuando ConfigContext carga la configuracion actualizada del backend, actualiza las variables CSS y las persiste en localStorage. Tercero, en visitas subsecuentes, el script inline del paso uno aplica los colores cacheados instantaneamente, eliminando cualquier parpadeo visual.

8. Flujo Completo de un Pedido

Para entender como interactuan todos los componentes, es util seguir el ciclo de vida completo de un pedido desde la perspectiva de cada actor del sistema.

8.1 Creacion del Pedido

El comprador agrega productos al carrito desde el frontend web o la app movil, selecciona los items y completa el checkout. El cliente envia un POST a /orders con los detalles del pedido (items, direccion de entrega, metodo de pago). El backend crea el registro de Order con sus OrderItems asociados en MongoDB, estableciendo el estado inicial como pending. El comprador queda suscrito al canal Pusher private-user-{userId} para recibir notificaciones sobre su pedido.

8.2 Asignacion y Preparacion

El administrador recibe la notificacion del nuevo pedido a traves del evento Pusher emitido por el backend. Desde el panel de administracion, puede cambiar el estado del pedido a confirmed. Si el sistema opera en modo multi-store, la asignacion se distribuye automaticamente entre las tiendas disponibles. Una vez confirmado, el pedido se asigna a un repartidor, quien recibe la notificacion en JO-Delivery y puede aceptar la entrega.

8.3 Entrega en Tiempo Real

Una vez que el repartidor acepta la entrega, inicia el GPS tracking llamando a startTracking(orderId) desde JO-Delivery. Las coordenadas se envian cada 15 segundos al backend, que las reenvia a traves de Pusher al canal private-order-{orderId}. El comprador puede ver la ubicacion del repartidor en la pagina de tracking (/tracking/[id]) del frontend web. El mapa se actualiza automaticamente cada vez que llegan nuevas coordenadas. Durante toda la entrega, comprador y repartidor pueden comunicarse a traves del chat del pedido.

8.4 Finalizacion del Pedido

Cuando el repartidor llega al destino, puede cambiar el estado a delivered desde la app. El backend actualiza el registro de Order y emite notificaciones Pusher al comprador y al administrador. El comprador puede calificar la experiencia y dejar un comentario. El historial del pedido queda disponible en la seccion de pedidos de todas las plataformas. El GPS tracking se detiene automaticamente al llamar a stopTracking().

9. Seguridad y Buenas Practicas

La seguridad del sistema se implementa en multiples capas. La autentificacion por JWT asegura que solo usuarios validos puedan acceder a los endpoints protegidos. La cabecera X-Platform permite al backend verificar la procedencia de cada peticion. Los endpoints de administracion requieren el rol admin. Los archivos subidos se validan por tipo MIME y tamano (maximo 2MB). Las contraseñas se hashean antes de almacenarlas. La BACKEND_SERVICE_PASSWORD protege endpoints internos entre servicios.

Pusher utiliza canales privados que requieren autorizacion del backend, impidiendo que usuarios no autenticados se suscriban a canales de otros usuarios. El sistema tambien implementa una separacion clara de datos por plataforma, asegurando que un repartidor solo pueda ver los pedidos asignados a el y un comprador solo pueda ver sus propios pedidos. Las configuraciones sensibles se almacenan como variables de entorno en Vercel y nunca se exponen al cliente.